Approximation algorithms

We are in the business of designing efficient algorithms.

Over the last few weeks we are Struggling or going through a bad Phase in our business.

As we have seen,

Most of the interesting Optimization Problems are NP-hard.

Thus, unless P = NP, there are no efficient Algorithms to find optimal solutions to such Problems.

Q What can we do in this case?

If $P \neq NP$,

we can't simultaneously have algorithms that

① always finds optimal solutions

② run in polynomial time

③ work for all instances

We must relax at least one of the above requirements to deal with NP-hard problems.

Suppose we relax condition ③ "for any instance"

ie, we find Polynomial time algorithms for special cases of the Problem. [Solve only some instances]

This is useful when instances one desire to solve fall into one of these special cases.

Suppose we relax condition ② "Polynomial-time"

This is useful if someone willing to wait longer, to find the best possible solution.

It may happen that sometimes, one is never certain that for the next input encountered, the algorithm will terminate in any reasonable amount of time.

[Exact exponential-time algorithms]

Finally

Suppose we relax condition (1) "Optimal Solution"

By far this is the most common approach.

[Approximation Algorithms]

We can try to give our algorithm a little freedom

Instead of demanding that the algorithm finds the best possible (optimal) solution, we will allow the algorithm to find a solution that is within a certain range of the optimal solution.

As usual we demand: algorithm should run in polynomial time.

Approximation Algorithms

optimizations

Many Problems of Practical Significance are NP-hard we don't know how to find an optimal solution in Polynomial time.

However we Still want a soln that is as good as possible even if it is not the optimal answer.

In Practice, near-optimal solutions are often good enough

We call an algorithm that returns near-optimal solutions an approximation algorithm

We Study Polynomial time approximation algorithms for several NP-hard optimization problems.

Key Question:

How to argue | prove that an algorithm finds near-optimal solution to a Problem?

Approximation Quality

- Constant factor approximation
- Polynomial time approximation schemes

Terminology

Assume that all solns have positive cost

Let Π be an optimization problem (Say minimization)

we say that an algorithm solving Π has an approximation ration $\rho(n)$ if for any input of size n, the cost C of the soln produced by the algorithm is within a factor of $\rho(n)$ of the cost C^* of an optimal solution.

ie, $$\frac{C}{C^*} \leq \rho(n)$$

[Called $\rho(n)$ - approximating algorithm]

ie, $$0 < C^* \leq C$$

Similarly we can define, for maximization problems

$$\frac{C}{C^*} \geq \frac{1}{\rho} \quad \text{or} \quad C \geq \frac{1}{\rho} C^*$$

ie., $$0 < C \leq C^*$$

If $\rho(n)$ is constant then we call the algorithm as constant factor approximation algorithm.

Defn (Formal):

(i) An Algorithm ALG for a **minimization Problem** is called a ρ-approximation algorithm for some $\rho > 1$, if $ALG(I) \leq \rho \cdot OPT(I)$ for all inputs I

(ii) An Algorithm ALG for a **Maximization Problem** is called a ρ-approximation algorithm for some $\rho > 1$, if $ALG(I) \geq \frac{1}{\rho} OPT(I)$ for all inputs I

1-approximation algorithm produces an optimal soln and an approximation algorithm with a large approximation ratio may return a soln that is much worse than optimal.

* When designing an approximation algorithm for an NP-hard optimization problem, we need to establish the approximation guarantee. That is the cost of the solution produced by the algorithm needs to be compared with the cost of an optimal solution.

* But we know that computing the cost of an optimal solution is a difficult task.

Q: How do we establish the approximation guarantee?

we will demonstrate with an example.

VERTEX COVER PROBLEM

Suppose A is a factor 2-approximation algorithm for Vertex cover Problem and A returns Vertex Cover with 100 vertices on input a graph G.

Q1: What is the maximum number of vertices in Optimal solution ?

Q2: What is the minimum number of vertices in Optimal solution ?

Problem 2 :

Same as above except replace VertexCover Problem with Clique Problem.

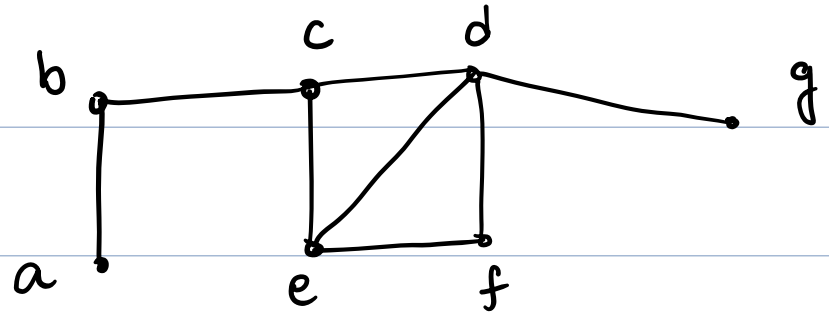
Vertex cover Problem

Recap: A vertex cover of an undirected graph G, is a subset $X \subseteq V(G)$ such that if uv is an edge in G, then either $u \in X$ or $v \in X$ (or both).

The size of a vertex cover is the number of vertices in it.

The vertex cover problem is to find a vertex cover of minimum size in a given undirected graph.

Example:



$X = \{b, d, e\}$ is a vertex cover of size 3.

Algorithm

APPROX-VC(G)

1 $X = \phi$

2 $E' = E(G)$

3 while $E' \neq \phi$

4 let uv be an arbitrary edge of E'

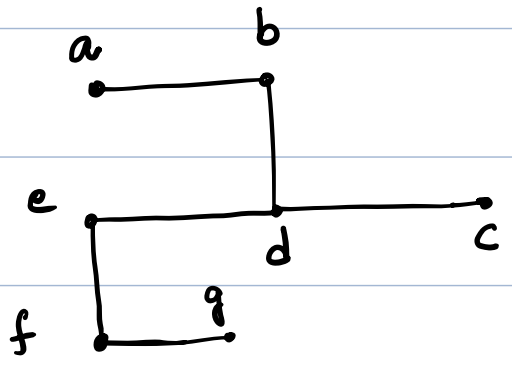
5 $X = X \cup \{u, v\}$

6 remove from E' every edge incident on either u or v

7 return X

Running time : $O(V+E)$ using adjacency lists to represent E'.

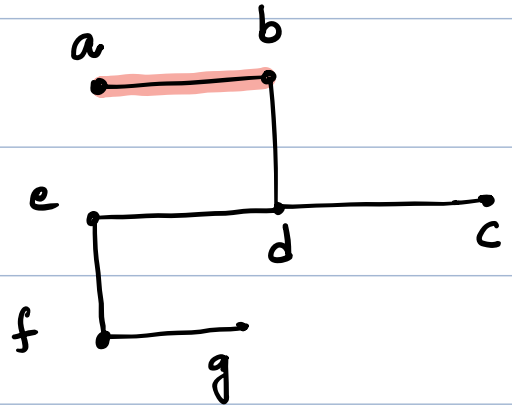
Example:



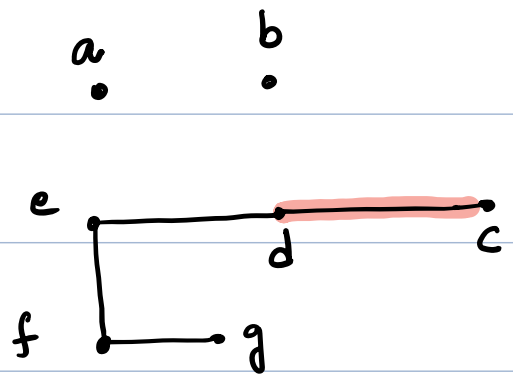
Graph G

Let us run APPROX-VC on the above graph

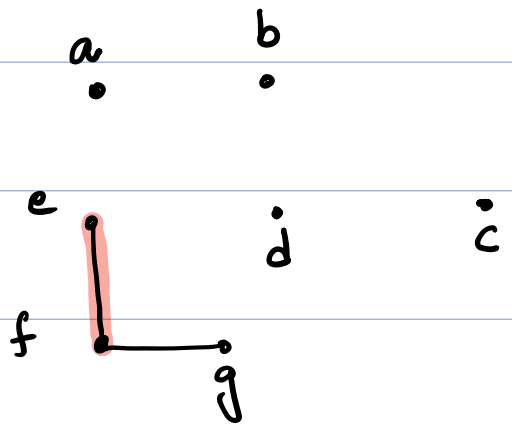
$X = \phi$



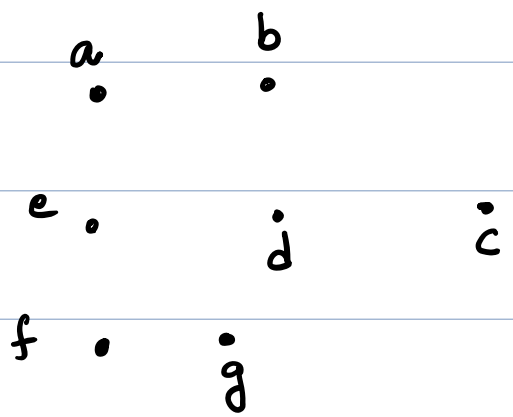
$X = \{a, b\}$



$X = \{a, b, c, d\}$



$X = \{a, b, c, d, e, f\}$



Soln returned by algo is $X = \{a, b, c, d, e, f\}$

Claim: APPROX-VC is a Polynomial-time 2-approximation algorithm.

Proof: Clearly the set of vertices returned by the algorithm is a vertexcover.

Let F denote the set of edges that line 4 of algorithm Picked.

In order to cover the edges in F, any optimal vertex cover X^* must include at least one endPoint of each edge in F.

No two edges in F share an endpoint, since once an edge is Picked in line 4, all other edges that are incident on its endpoints are deleted from E' in line 6.

∴ No two edges in F are covered by the same vertex from X^*, and we get

$$|X^*| \geq |F| \quad \text{— (1)}$$

Each execution of line 4 pick an edge for which neither endpoints is already in X, giving an upperbound on X.

$$|X| = 2|F| \quad — (2)$$

From (1) & (2)

$$|X| = 2|F| \leq 2|X^*|$$

ie, algorithm return a vertex cover X whose size is guaranteed to be no more than twice the size of an optimal vertex cover.

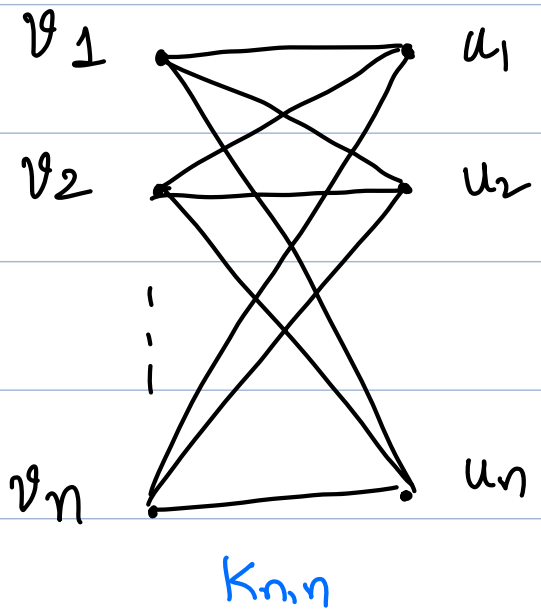
Note: under Unique games Conjecture, this factor of 2 for the VertexCover Problem is the best possible one.

Q: Can the approximation guarantee of above algorithm be improved by a better analysis?

Ans: No,

We can show that 2-approximation is a tight bound by a tight example.

Consider the graph $K_{n,n}$ (Complete bipartite graph)



$K_{n,n}$

"Any infinite family of instances showing that the analysis of an approximation algorithm is tight is called tight example"

The 2-approximation algorithm selects n edges hence returns a Vertex cover of size $2n$.

But clearly the optimal solution $= n$

Load Balancing Problem

Given a set of m identical machines $M_1, \ldots M_m$ and a set of n jobs, each job j has a processing time t_j. A machine can process at most one job at a time.

We seek to assign each job to one of the machines so that loads placed on all machines are as "balanced" as possible.

More precisely, in any assignment of jobs to machines, we can let $A(i)$ denote the set of jobs assigned to machine M_i. Then the total time machine M_i needs to work for a total time

$$T_i = \sum_{j \in A(i)} t_j$$

We call T_i as the load on machine M_i.

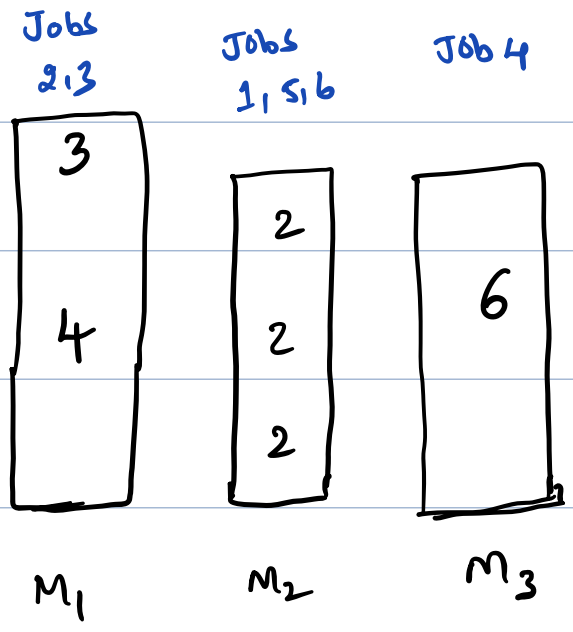
We seek to minimize a quantity known as "makespan" (T). which is the maximum load on any machine. $T = \max_i T_i$.

Remark: Finding an assignment of minimum makespan is NP-hard.

Example:

	M_1	M_2	M_3

Jobs:	1	2	3	4	5	6
Processing time:	2	3	4	6	2	2



$T_1 = 7$, $T_2 = 6$ $T_3 = 6$

Makespan $T = 7$

[Warmup]

If we have fewer than m jobs then can we solve load balancing efficiently?

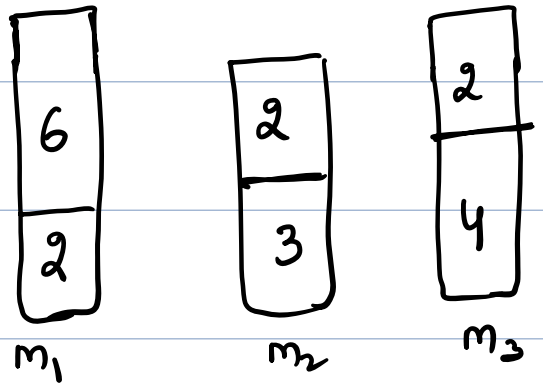
Algorithm

Idea: [Greedy Algorithm]

The Algorithm makes one pass through the jobs in any order, when it comes to job j, it assigns j to the machine whose load is smallest so far.

```
Greedy-Balance:
Start with no jobs assigned
Set T_i = 0 and A(i) = ∅ for all machines M_i
For j = 1, ..., n
  Let M_i be a machine that achieves the minimum min_k T_k
  Assign job j to machine M_i
  Set A(i) ← A(i) ∪ {j}
  Set T_i ← T_i + t_j
EndFor
```

Let's run the above algorithm one our previous example with the sequence 2, 3, 4, 6, 2, 2



Makespan = 8

↓

Clearly not optimal.

let T denote the makespan obtained by the algorithm.

T^* is the minimum possible makespan (ie, optimal makespan)

Observations:

① $T^* \geq \frac{1}{m} \sum_j t_j$

ie, one of the m machines must to do at least a $\frac{1}{m}$ fraction of the total work.

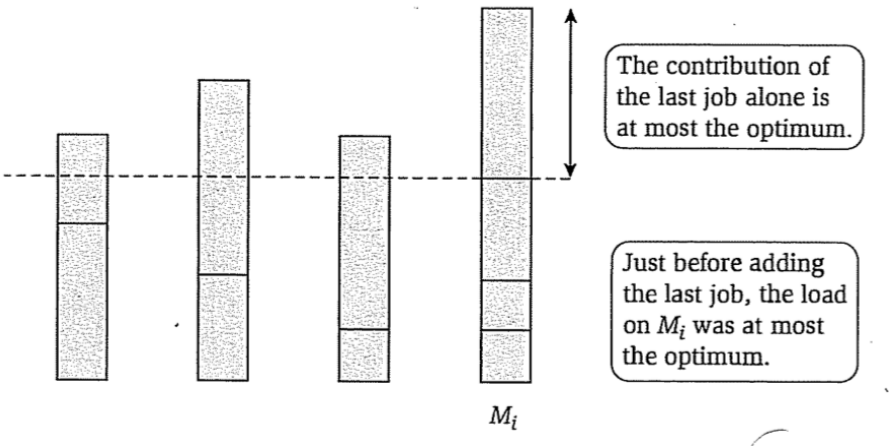
② $T^* \geq \max_j t_j$ [Trivial]

Claim: Algorithm Greedy-Balance Produces an assignment of jobs to machines with makespan $T \leq 2T^*$

Proof Idea:

Let M_i be the machine that attains the maximum load T in our assignment.

Let j be the last job assigned to M_i



When we assigned job j to M_i, the machine M_i had the smallest load of any machine.

$\therefore$ Load of machine M_i before assignment of job j was $T_i - t_j$

Also, at that moment every machine had load at least $T_i - t_j$.

$$\therefore \quad \sum_k T_k \geq m(T_i - t_j)$$

$$T_i - t_j \leq \frac{1}{m} \sum_k T_k$$

$$= \frac{1}{m} \sum_j t_j$$

$$= T^* \quad \text{(from observation 1)}$$

ie, $T_i - t_j \leq T^*$ — (A)

Now we bound the remaining part of the load on M_i (ie, t_j).

From observation (2), we have $t_j \leq T^*$. — (B)

From (A) & (B)

$$T_i = (T_i - t_j) + t_j$$

$$\leq T^* + T^* = 2T^*.$$

Since our makespan $T = T_i \leq 2T^*$.

Q: Is our analysis tight.

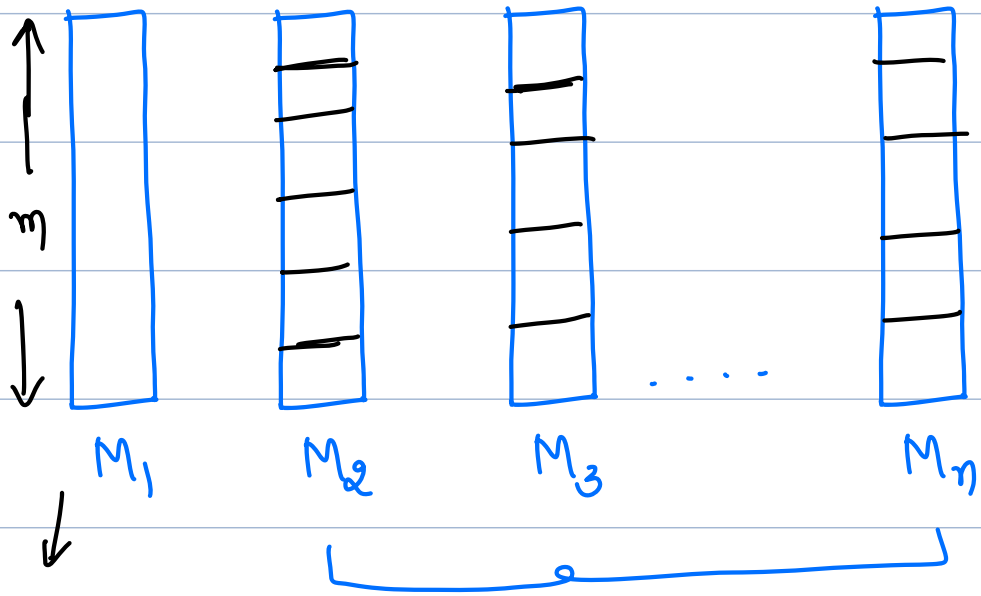
YES, see the below example.

Eg: m – machines

$m(m-1)$ jobs of length 1

one job of length m

Optimal soln:

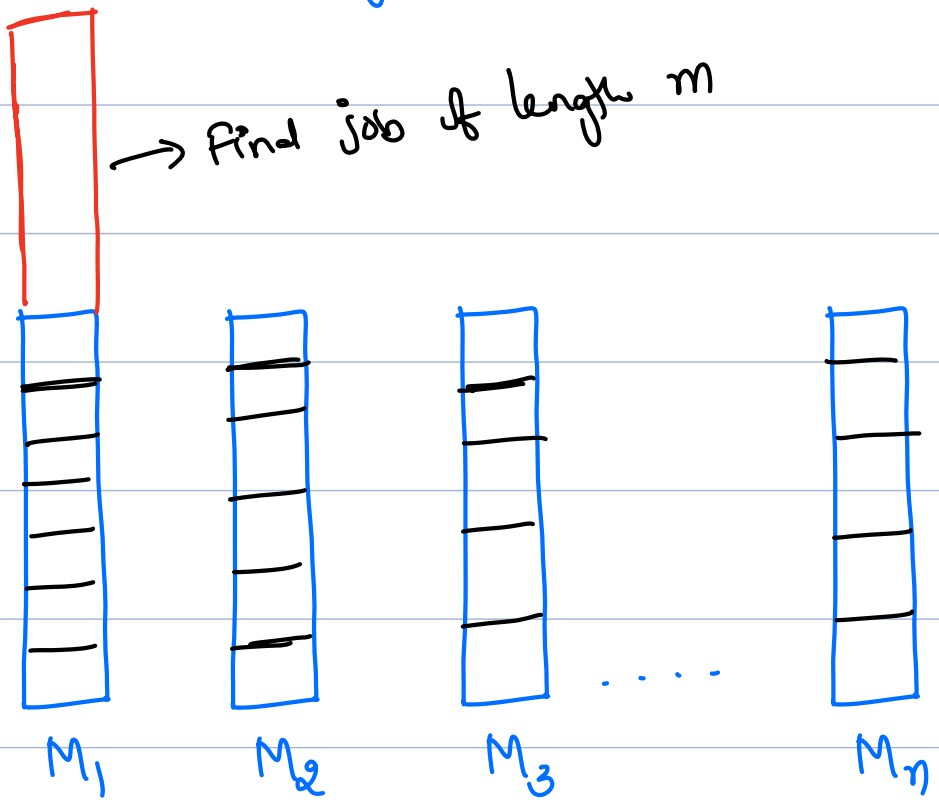


Assigned one job of length m

$(m-1)$ machines each assigned m jobs of length 1

Makespan $= m$

Output of the Algorithm:



Makespan $= (m-1) + m = 2m-1$

Improved Approximation Algorithm

In the previous algorithm, we may encounter the following bad case.

We spread everything out very evenly across the machines and then one last, giant, unfortunate job arrived.

Intuitively, it looks like it would help to get the largest jobs arranged nicely first, with the idea that later, small jobs can only do so much damage.

Algorithm [Largest Processing Time Rule Algorithm]

Sorted-Balance:
Start with no jobs assigned
Set $T_i = 0$ and $A(i) = \emptyset$ for all machines M_i
Sort jobs in decreasing order of processing times t_j
Assume that $t_1 \geq t_2 \geq \ldots \geq t_n$
For $j = 1, \ldots, n$
  Let M_i be the machine that achieves the minimum $\min_k T_k$
  Assign job j to machine M_i
  Set $A(i) \leftarrow A(i) \cup \{j\}$
  Set $T_i \leftarrow T_i + t_j$
EndFor

Analysis: **Recap:** We have m machines if we have at most m jobs then the above algorithm solution will clearly be optimal.

Observation-3: If there are more than m jobs then

$$T^* \geq 2\, t_{m+1}$$

Proof: look at the first $m+1$ jobs in the sorted order. they each take $\geq t_{m+1}$ time.

There are $m+1$ jobs & m machines. So there must be a machine that gets assigned two of these jobs. This machine will have processing time $\geq 2\, t_{m+1}$.

* $T^* \geq 2\, t_j$ for every $j \geq m+1$

Claim: Algorithm Sorted-Balance produces an assignment of jobs to machines with makespan $T \leq \frac{3}{2} T^*$

Proof:- Analysis is similar to the previous algorithm.

Let M_i be the machine that has the maximum load. If M_i has assigned a single job then the schedule is optimal.

So assume M_i has at least two jobs.

Let j be the last job assigned to the machine M_i.

Clearly $j \geq m+1$.

Since the algorithm will assign the first m jobs to m distinct machines.

Thus $t_j \leq \frac{1}{2} T^*$ (from observation 3)

From here onward Proof is similar to previous algo analysis Proof

We had

$$T_i - t_j \leq T^* \quad \& \quad t_j \leq T^* \quad \text{— (A)}$$

But now we have $t_j \leq \frac{1}{2} T^*$ — (B)

From (A) & (B)

$$T_i \leq \frac{3}{2} T^*$$

hence makespan $T \leq \frac{3}{2} T^*$.

Note: Our analysis is not tight.

A better analysis will show that above algorithm is a $\frac{4}{3}$ approximation algorithm.

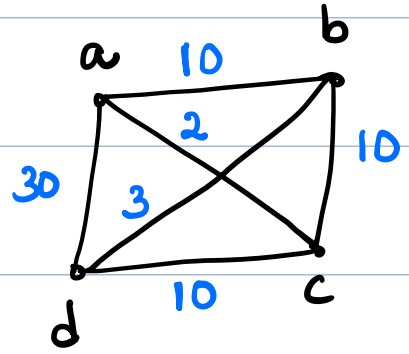
Travelling Salesman Problem

We are given a undirected complete graph G that has a non-negative integer cost c_{uv} associated with each edge $uv \in E(G)$, and we must find a hamiltonian cycle (a tour) of G with minimum cost.

For $A \subseteq E(G)$

$$c(A) = \sum_{uv \in A} c_{uv}$$

Example:



Hamiltonian cycle of minimum cost is

$a - b - d - c - a$ of cost $10 + 3 + 10 + 2 = 25$

Decision Version of TSP

I/p: Complete graph G, $c : V \times V \to \mathbb{Z}$, $k \in \mathbb{Z}$

Q: Does G has a Hamiltonian cycle with cost at most k.

Remark: TSP is NP-Complete

Proof Idea: Hamiltonian cycle $\leq_p$ TSP

Let $G = (V, E)$ be an instance of Hamiltonian cycle Problem. We construct an instance of TSP as follows.

We form a Completegraph $G' = (V, E')$, where $E' = \{ uv \mid u \in V, v \in V \}$. Cost function c is defined as

$$c_{uv} = \begin{cases} 0 & \text{if } uv \in E \\ 1 & \text{otherwise} \end{cases}$$

The instance of TSP is then $(G', c, 0)$ which can be constructed in Polynomial time.

Claim: The graph G has a hamiltonian cycle if and only if graph G' has tour of cost at most Zero.

Proof: Exercise

Therefore we now study a 2-approximation algorithm for TSP (special case).

Assumption : [Metric TSP]

We assume that the cost function c satisfies the triangle inequality.

ie, for all $u, v, w \in V(G)$

$$c_{uw} \leq c_{uv} + c_{vw}$$

if costs on edges of G satisfies the triangle inequality then it is called **Metric TSP**.

Theorem: Metric TSP is NP-hard

Proof Hamiltonian cycle $\leq_p$ Metric TSP

Given a graph G, construct a complete graph G' on the vertex set $V(G)$, with cost function c defined as follows.

$$c_{uv} = \begin{cases} 1 & \text{if } uv \in E \\ 2 & \text{otherwise} \end{cases}$$

Clearly c satisfies triangle inequality.

Claim: The graph G has a hamiltonian cycle if and only if graph G' has tour of cost n.

Proof: Exercise

2-approximation algorithm for metric TSP

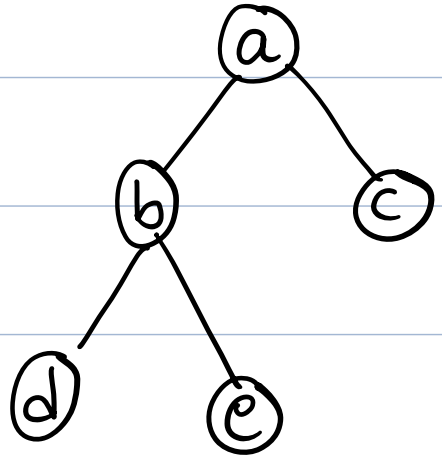
APPROX-TSP-TOUR(G, c)

1 select a vertex $r \in G.V$ to be a "root" vertex
2 compute a minimum spanning tree T for G from root r
  using MST-PRIM(G, c, r)
3 let H be a list of vertices, ordered according to when they are first visited
  in a preorder tree walk of T
4 **return** the hamiltonian cycle H

Recap: Tree Traversals

↓

walking the tree



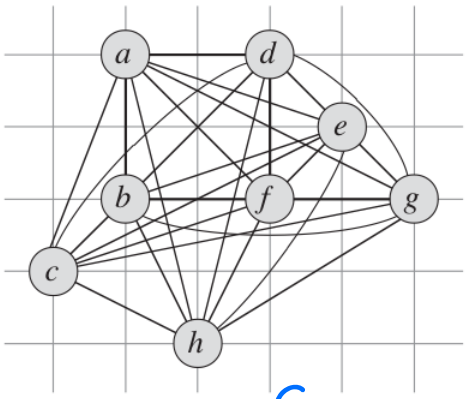
Inorder (left, root, right) : d b e a c

Preorder (root, left, right) : a b d e c

Postorder (left, right, root) : d e b c a

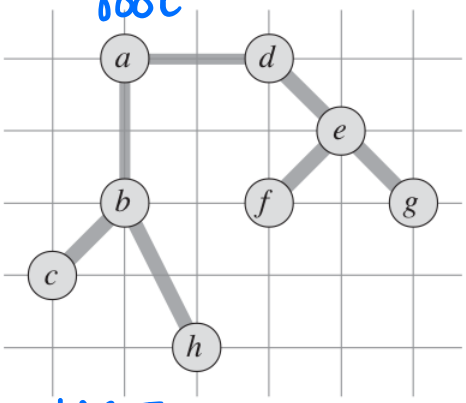
Illustration of APPROX-TSP-TOUR

Cost function is Euclidean distance

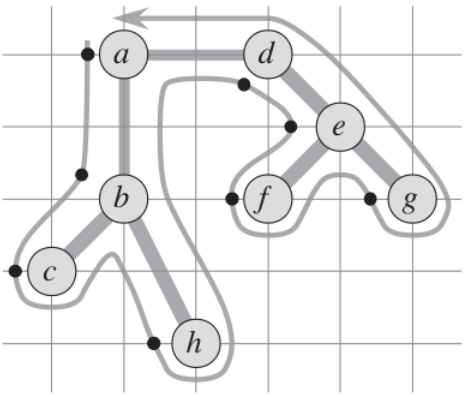


(a) G

root



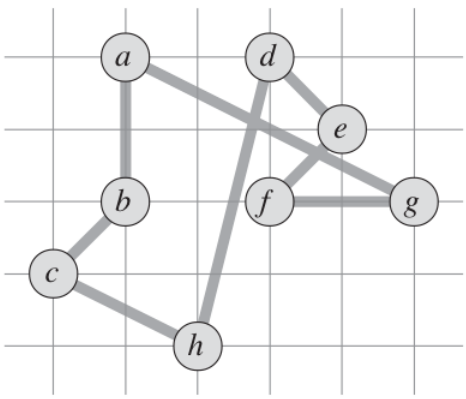
MST (b)



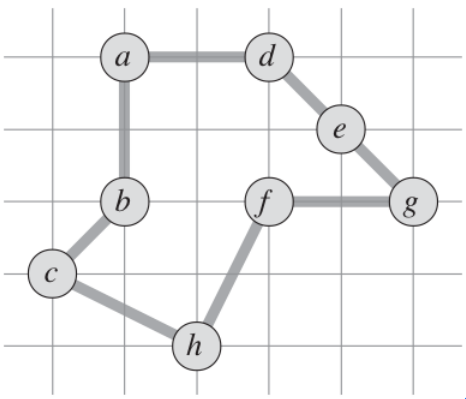
(c)

Walk of MST

a-b-c-b-h-
b-a-d-e-f-
e-g-e-d-a



(d)



(e) Optimal tour

Tour obtained by visiting the vertices in the order given by Preorder walk

(O/P of the APPROX-TSP-TOUR)

Running time of APPROX-TSP-TOUR

APPROX-TSP-TOUR(G, c)

1 select a vertex $r \in G.V$ to be a "root" vertex $O(1)$

2 compute a minimum spanning tree T for G from root r } Prim's algorithm $O(n^2)$
using MST-PRIM(G, c, r)

3 let H be a list of vertices, ordered according to when they are first visited in a preorder tree walk of T $O(n)$

4 **return** the hamiltonian cycle H $O(1)$

Analysis :

Let H^* denote an optimal tour for the given set of vertices.

We obtain a spanning tree by deleting any edge from a tour and each edge cost is non-negative.

∴ the weight of the MST T computed in line 2 of the algorithm provides a lower bound on the cost of an optimal tour.

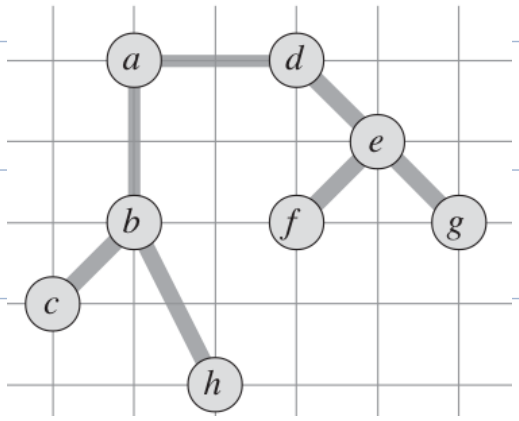
$$C(T) \leq C(H^*) \quad \text{—— (I)}$$

Full walk W of T list the vertices when they are first visited and also whenever they are returned to after a visit to a subtree.

Full walk in the example is

a - b - c - b - h - b - a - d - e - f -

e - g - e - d - a



ie, full walk traverses every edge of T exactly twice

$$\therefore \quad C(W) = 2\,C(T) \quad \text{— (II)}$$

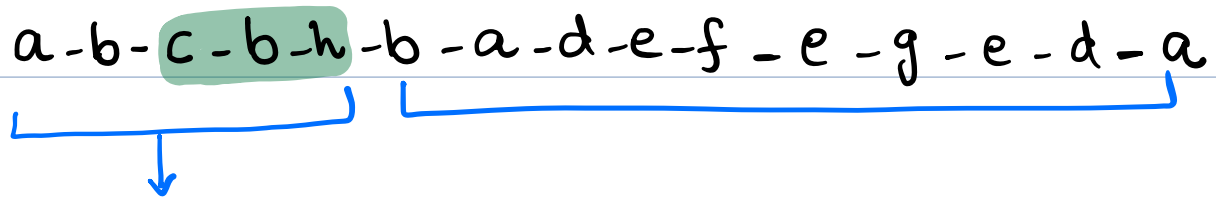
From (I) & (II)

$$C(W) \leq 2\,C(H^*) \quad \text{— (III)}$$

Unfortunately, the full walk W is generally not a tour, since it visits some vertices more than once.

Using the triangle inequality, we can delete a visit to any vertex from W and the cost does not increase.

Example



a - b - c - h

$c_{ch} \leq c_{cb} + c_{bh}$

By repeatedly applying this operation, we can remove from W all but the first visit to each vertex.

In our running example, we get

a - b - c - h - d - e - f - g

↳ Pre-order walk of the tree T.

Let H be the cycle corresponding to this Preorder walk. It is a Hamiltonian cycle as every vertex is visited exactly once.

Since H is obtained by deleting vertices from the full walk W, we have

$$c(H) \leq c(W) \quad \text{—(IV)}$$

from (III) & (IV)

we get $c(H) \leq 2\, c(H^*)$.

xx

Metric TSP $\frac{3}{2}$ approximation algorithm

First we recap few definitions.

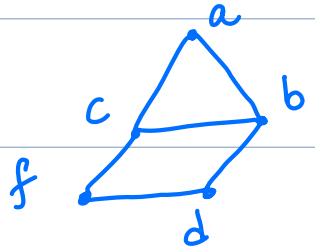
Def:- A **walk** in a graph is a sequence of vertices $v_1, v_2, \ldots v_k$ such that $v_i v_{i+1} \in E$ for $i = 1, 2, \ldots k-1$. (may not be ↑ distinct)

If the vertices in a walk are distinct then the walk is called a **Path**. If the edges in a walk are distinct then the walk is called a **trial**.

A **cycle** is a path $v_1, \ldots v_k$ together with the edge $v_k v_1$ $(k \geq 3)$.

A **Trial** that begins and ends at the same vertex is called a circuit (or) Closed trail

Ex



walk = a c f c b d

Trial = b a c b d

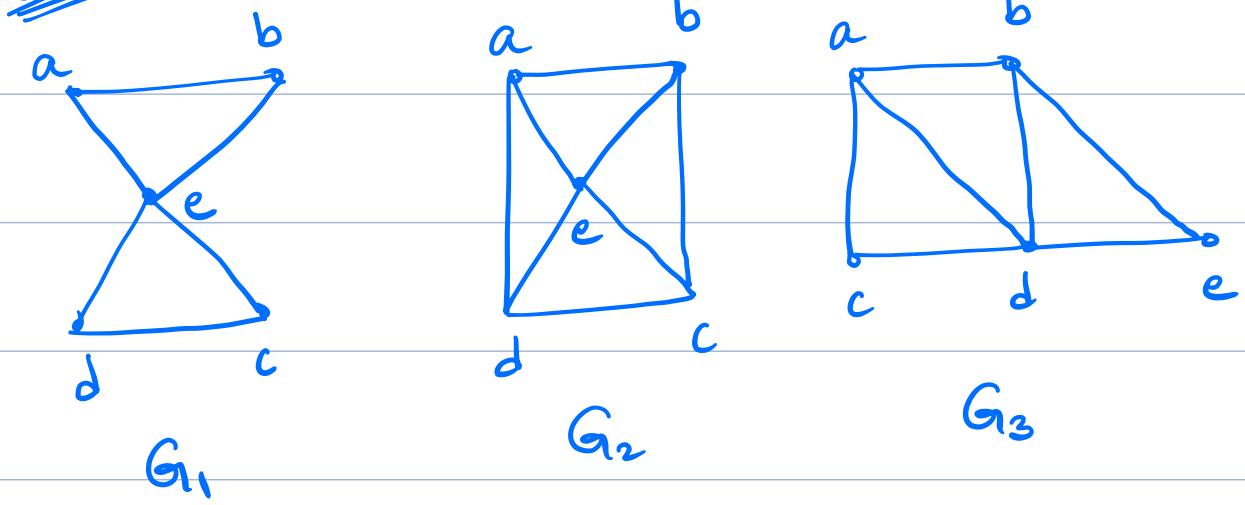
Path = a c f d

circuit = a c f d b a

Def:- A graph is **Eulerian** if it has a closed trail containing all edges. We call a closed trail a circuit when we do not specify the first vertex but keep the list in cyclic order.

An Eulerian circuit or Eulerian trail in a graph is a circuit or trail containing all the edges.

Example:



G_1 has an Euler circuit → a, e, c, d, e, b, a

G_2 & G_3 has no Euler circuit.

G_3 has an Euler trail → a, c, d, e, b, d, a, b

G_2 doesn't have Euler trail.

Definition:

A graph is called an **Eulerian graph** if it contains an Euler circuit.

Theorem:- (Euler 1736) A connected graph G is Eulerian iff every vertex has even degree.

Pmf: Skipped. Refer to any text on graph theory

Back to Metric TSP

* We can improve the approximation factor to 3/2 by adding a minimum cost Perfect matching on Vertices of odd degree in MST.

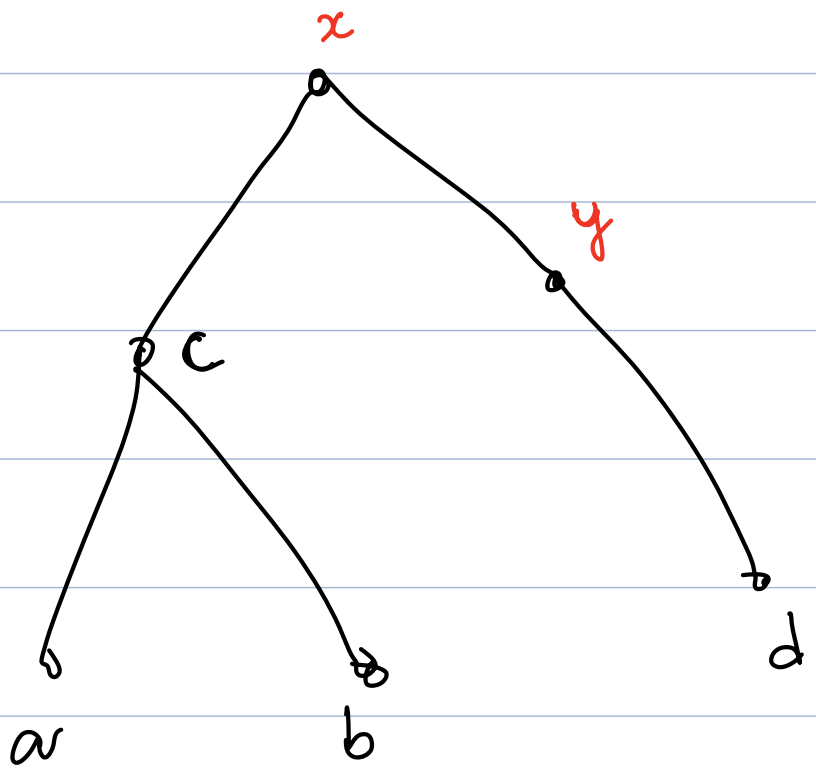
Algorithm 3.10 (Metric TSP – factor 3/2) [Christofides]

1. Find an MST of G, say T.
2. Compute a minimum cost perfect matching, M, on the set of odd-degree vertices of T. Add M to T and obtain an Eulerian graph.
3. Find an Euler tour, $\mathcal{T}$, of this graph.
4. Output the tour that visits vertices of G in order of their first appearance in $\mathcal{T}$. Let C be this tour.

Remark:

Number of odd-degree Vertices in a graph is even as Sum of degrees of all Vertices is even.

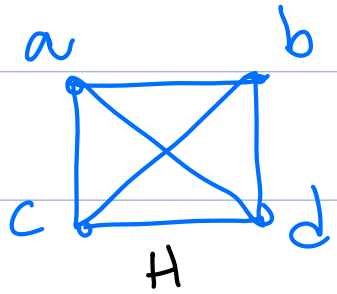
Idea:



MST T

{a, b, c, d} are odd degree Vertices.

Consider the subgraph induced by {a, b, c, d}



Find a Perfect matching M of minimum weight in H.

add the edges of M to T.

* The Algorithm that finds a Perfect matching of minimum weight is not discussed. We will take it as blackbox.

The Algorithms runs in $O(n^3)$ time.

Analysis:

Let V' denote the set of odd degree vertices in MST T.

Lemma: Let $V' \subseteq V$, such that $|V'|$ is even and let M be a minimum cost perfect matching on V'. Then, $\text{cost}(M) \leq OPT/2$

Proof:

Consider an optimal TSP tour of G, say L, let L' be the tour on V' obtained by short-cutting L. By the triangle inequality,

$$\text{cost}(L') \leq \text{cost}(L).$$

Observe, L' is the union of two perfect matchings on V', each consisting of alternate edges of L'.

Thus the cheaper of these matchings has

$$\text{cost} \leq \text{cost}(L')/2 \leq OPT/2$$

Hence, the cost of minimum weight P.M is atmost OPT/2.

$=$ (o/p of algo) $\uparrow$ MST $\uparrow$ matching

$$\text{Cost}(T') \leq \text{cost}(T) + \text{Cost}(M)$$

$$\leq OPT + OPT/2$$

$$= \frac{3}{2} OPT.$$

Note:

① The above $\frac{3}{2}$-algorithm to metric TSP is the best known approximation algo as of now (recently some progress has been made)

② Finding a better approximation algorithm for metric TSP (triangle inequality holds) is one of the outstanding open problems in this area.

* It is NP-hard to approximate metric TSP within a factor of $\frac{123}{122}$ [KLS15]

Theirs is a randomised algorithm and it follows similar principles to Christofides' algorithm, but uses a randomly chosen tree from a carefully chosen random distribution in place of the minimum spanning tree. The paper was published at STOC'21 where it received a best paper award. [Taken from Wikipedia]

Q: What happens if we drop the assumption that cost function c satisfies the triangle inequality ? Answer is given in the next Page.

Theorem: If $P \neq NP$, then for any constant $\rho \geq 1$ there is no Polynomial-time approximation algorithm with approximation ratio ρ for the general TSP.

i.e, There is no constant factor approximation algorithm to TSP unless $P = NP$.

Proof:- Proof by contradiction,

Suppose that for some constant $\rho \geq 1$, there is a Polynomial-time approximation algorithm A with approximation ratio ρ. WLOG, assume that ρ is an integer.

ie, $C(A) \leq \rho \, C(OPT)$, $\rho \geq 1$

$C(A)$ → cost of o/p of A

We show how to use A to solve Hamiltonian-cycle Problem in Polynomial time. As Ham.cycle is NP-complete, we get $P = NP$.

Let $G=(V,E)$ be an instance of H. cycle Problem.

we build G' an instance of TSP from G as follows.

$G'=(V,E')$ be the Complete graph on V

ie, $E'=\{uv \mid u,v \in V,\ u \neq v\}$

Cost to each edge in E' is defined as

$$c_{uv}=\begin{cases} 1 & \text{if } uv \in E \\ p|V|+1 & \text{otherwise} \end{cases}$$

* c doesn't satisfy triangle inequality.

Clearly we can construct G' and c from G in Polynomial time.

Now consider the TSP (G',c).

Observe that

"If G has a Hamiltonian cycle then min cost TS Tour $= |V|$"

If G doesn't have Hamiltonian cycle then min cost TS tour contain at least one new edge. So cost is

$$\begin{aligned} \text{at least} &\geq |V| - 1 + \rho|V| + 1 \\ &= \rho|V| + |V| \\ &> \rho|V| \end{aligned}$$

Because edges not in G are so costly, there is a gap of at least $\rho|V|$ b/w the cost of a tour that is a hamiltonian cycle in G and the cost of any tour.

∴ the cost of a tours that is not a hamiltonian cycle in G is at least a factor of $\rho + 1$ greater than the cost of a tour that is a Hamiltonian cycle in G.

Run the ρ-approximation algorithm A on G'. if G has Hamiltonian cycle it will o/p a tour with cost $\leq \rho|V|$

otherwise A returns a tour of cost more than $\rho|V|$.

$\therefore$ we can use A to solve Hamiltonian cycle in Polynomial time.

Note that in the above Reduction edge costs violate the triangle inequality.

In its full generality, TSP cannot be approximated assuming $P \neq NP$.